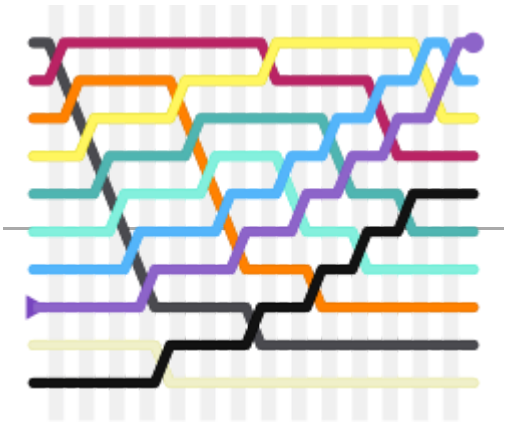
WIKIPEDIA 25
25 years of the free encyclopedia

Bubble sort

Bubble sort, sometimes referred to as **sinking sort**, is a simple sorting algorithm that repeatedly steps through the input list element by element, comparing the current element with the one after it, swapping their values if needed. These passes through the list are repeated until no swaps have to be performed during a pass, meaning that the list has become fully sorted. The algorithm, which is a comparison sort, is named for the way the larger elements "bubble" up to the top of the list.

It performs poorly in real-world use and is used primarily as an educational tool. More efficient algorithms such as quicksort, timsort, or merge sort are used by the sorting libraries built into popular programming languages such as Python and Java.[2][3]

Bubble sort



Static visualization of bubble sort[1]

Class	Sorting algorithm
Data structure	Array
Worst-case performance	$O(n^2)$ comparisons, $O(n^2)$ swaps
Best-case performance	$O(n)$ comparisons, $O(1)$ swaps
Average performance	$O(n^2)$ comparisons, $O(n^2)$ swaps
Worst-case space complexity	$O(n)$ total, $O(1)$ auxiliary
Optimal	No

History

The earliest description of the bubble sort algorithm was in a 1956 paper by mathematician and actuary Edward Harry Friend,[4] *Sorting on electronic computer systems*,[5] published in the third issue of the third volume of the *Journal of the Association for Computing Machinery* (ACM), as a "Sorting exchange algorithm." Friend described the fundamentals of the algorithm, and although initially his paper went unnoticed, some years later it was rediscovered by many computer scientists, including Kenneth E. Iverson, who coined its current name.

Analysis

Performance

Bubble sort has a worst-case and average complexity of $O(n^2)$, where n is the number of items being sorted. Most practical sorting algorithms have substantially better worst-case or average complexity, often $O(n \log n)$. Even other $O(n^2)$ sorting algorithms, such as insertion sort, generally run faster than bubble sort, and are no more complex. For this reason, bubble sort is rarely used in practice.

Like insertion sort, bubble sort is adaptive, which can give it an advantage over algorithms like quicksort. This means that it may outperform those algorithms in cases where the list is already mostly sorted (having a small number of inversions), despite the fact that it has worse average-case time complexity. For example, bubble sort is $O(n)$ on a list that is already sorted, while quicksort would still perform its entire $O(n \log n)$ sorting process.

While any sorting algorithm can be made $O(n)$ on a presorted list simply by checking the list before the algorithm runs, improved performance on almost-sorted lists is harder to replicate.

An example of bubble sort. Starting from the beginning of the list, compare every adjacent pair, swapping their position if they are not in the right order (the latter one is smaller than the former one). After each iteration, one less element (the last one) needs to be compared until there are no more elements left to be compared.

Rabbits and turtles

The distance and direction that elements must move during the sort determine bubble sort's performance because elements move in different directions at different speeds. An element that must move toward the end of the list can move quickly because it can take part in successive swaps. For example, the largest element in the list will win every swap, so it moves to its sorted position on the first pass even if it starts near the beginning. On the other hand, an element that must move toward the beginning of the list cannot move faster than one step per pass, so elements move toward the beginning very slowly. If the smallest element is at the end of the list, it will take $n - 1$ passes to move it to the beginning. This has led to these types of elements being named rabbits and turtles, respectively, after the characters in Aesop's fable of The Tortoise and the Hare.

Various efforts have been made to eliminate turtles to improve the speed of bubble sort. Cocktail sort is a bi-directional bubble sort that goes from beginning to end, and then reverses itself, going end to beginning. It can move turtles fairly well, but it retains $O(n^2)$ worst-case complexity. Comb sort compares elements separated by large gaps, and can move turtles extremely quickly before proceeding to smaller and smaller gaps to smooth out the list. Its average speed is comparable to faster algorithms like quicksort.

Step-by-step example

Take an array of numbers "5 1 4 2 8", and sort the array from lowest number to greatest number using bubble sort. In each step, elements written in **bold** are being compared. Three passes will be required:

First pass

(**5 1** 4 2 8) → (**1 5** 4 2 8), Here, algorithm compares the first two elements, and swaps since 5 > 1.
(1 **5 4** 2 8) → (1 **4 5** 2 8), Swap since 5 > 4
(1 4 **5 2** 8) → (1 4 **2 5** 8), Swap since 5 > 2
(1 4 2 **5 8**) → (1 4 2 **5 8**), Now, since these elements are already in order (8 > 5), algorithm does not swap them.

Second pass

(**1 4** 2 5 8) → (**1 4** 2 5 8)
(1 **4 2** 5 8) → (1 **2 4** 5 8), Swap since 4 > 2

(1 2 **4 5** 8) → (1 2 **4 5** 8)
(1 2 4 **5 8**) → (1 2 4 **5 8**)

Now, the array is already sorted, but the algorithm does not know if it is completed. The algorithm needs one additional **whole** pass without **any** swap to know it is sorted.

Third pass

(**1 2** 4 5 8) → (**1 2** 4 5 8)
(1 **2 4** 5 8) → (1 **2 4** 5 8)
(1 2 **4 5** 8) → (1 2 **4 5** 8)
(1 2 4 **5 8**) → (1 2 4 **5 8**)

Implementation

Pseudocode implementation

In pseudocode the algorithm can be expressed as (0-based array):

```
procedure bubbleSort(A : list of sortable items)
    n := length(A)
    repeat
        swapped := false
        for i := 1 to n-1 inclusive do
            { if this pair is out of order }
            if A[i-1] > A[i] then
                { swap them and remember something changed }
                swap(A[i-1], A[i])
                swapped := true
            end if
        end for
    until not swapped
end procedure
```

Optimizing bubble sort

The bubble sort algorithm can be optimized by observing that the n-th pass finds the n-th largest element and puts it into its final place. So, the inner loop can avoid looking at the last $n - 1$ items when running for the n-th time:

```
procedure bubbleSort(A : list of sortable items)
    n := length(A)
    repeat
        swapped := false
        for i := 1 to n - 1 inclusive do
            if A[i - 1] > A[i] then
                swap(A[i - 1], A[i])
                swapped := true
            end if
        end for
        n := n - 1
    until not swapped
end procedure
```

More generally, it can happen that more than one element is placed in its final position on a single pass. In particular, after every pass, all elements after the last swap are sorted, and do not need to be checked again. This allows us to skip over many elements, resulting in about a 50% improvement in the worst-case comparison count (though no improvement in swap counts), and adds very little complexity because the new code subsumes the `swapped` variable:

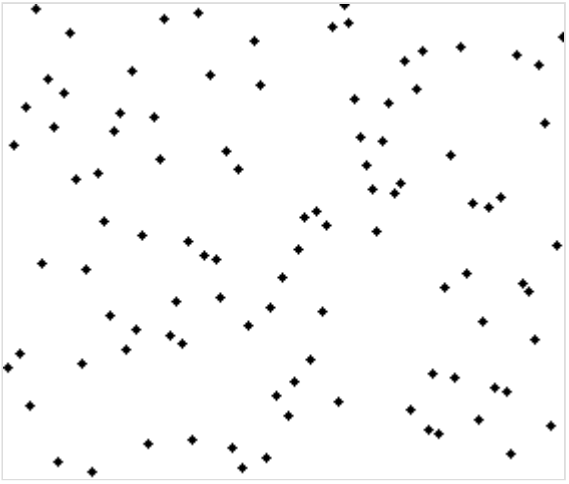
To accomplish this in pseudocode, the following can be written:

```
procedure bubbleSort(A : list of sortable items)
    n := length(A)
    repeat
        newn := 0
        for i := 1 to n - 1 inclusive do
            if A[i - 1] > A[i] then
                swap(A[i - 1], A[i])
                newn := i
            end if
        end for
        n := newn
    until n ≤ 1
end procedure
```

Alternate modifications, such as the cocktail shaker sort attempt to improve on the bubble sort performance while keeping the same idea of repeatedly comparing and swapping adjacent items.

Use

Although bubble sort is one of the simplest sorting algorithms to understand and implement, its $O(n^2)$ complexity means that its efficiency decreases dramatically on lists of more than a small number of elements. Even among simple $O(n^2)$ sorting algorithms, algorithms like insertion sort are usually considerably more efficient.



Bubble sort. The list was plotted in a Cartesian coordinate system, with each point (x, y) indicating that the value y is stored at index x. Then the list would be sorted by bubble sort according to every pixel's value. Note that the largest end gets sorted first, with smaller elements taking longer to move to their correct positions.

Due to its simplicity, bubble sort is often used to introduce the concept of an algorithm, or a sorting algorithm, to introductory computer science students. However, some educators such as Owen Astrachan have gone to great lengths to disparage bubble sort and its continued popularity in computer science education, recommending that it no longer even be taught.[6]

The Jargon File, which famously calls bogosort "the archetypical [sic] perversely awful algorithm", also calls bubble sort "the generic bad algorithm".[7] Donald Knuth, in *The Art of Computer Programming*, concluded that "the bubble sort seems to have nothing to recommend it, except a catchy name and the fact that it leads to some interesting theoretical problems", some of which he then discusses.[8]

Bubble sort is asymptotically equivalent in running time to insertion sort in the worst case, but the two algorithms differ greatly in the number of swaps necessary. Experimental results such as those of Astrachan have also shown that insertion sort performs considerably better even on random lists. For these reasons, many modern algorithm textbooks avoid using the bubble sort algorithm in favor of insertion sort.

Bubble sort also interacts poorly with modern CPU hardware. It produces at least twice as many writes as insertion sort, twice as many cache misses, and asymptotically more branch mispredictions. Experiments by Astrachan sorting strings in Java show bubble sort to be roughly one-fifth as fast as an insertion sort and 70% as fast as a selection sort.[6]

In computer graphics, bubble sort is popular for its capability to detect a very small error (like a swap of just two elements) in almost-sorted arrays and fix it with just linear complexity ($2n$). For example, it is used in a polygon filling algorithm, where bounding lines are sorted by their x coordinate at a specific scan line (a line parallel to the x axis), and with incrementing y, their order changes (two elements are swapped) only at intersections of two lines. Bubble sort is a stable sort algorithm, like insertion sort.

Variations

- Odd–even sort is a parallel version of bubble sort, for message-passing systems.
- Passes can be from right to left, rather than left to right. This is more efficient for lists with unsorted items added to the end.
- Cocktail shaker sort alternates leftwards and rightwards passes.

Debate over name

Bubble sort has been occasionally referred to as a "sinking sort".[9]

For example, Donald Knuth describes the insertion of values at or towards their desired location as letting "[the value] settle to its proper level", and that "this method of sorting has sometimes been called the *sifting* or *sinking* technique.[10]

This debate is perpetuated by the ease with which one may consider this algorithm from two different but equally valid perspectives:

1. The *larger* values might be regarded as *heavier* and therefore be seen to progressively *sink* to the *bottom* of the list
2. The *smaller* values might be regarded as *lighter* and therefore be seen to progressively *bubble up* to the *top* of the list.

In popular culture

In a 2007 interview, former Google CEO Eric Schmidt asked then-presidential candidate Barack Obama about the best way to sort one million integers; Obama paused for a moment and replied: "I think the bubble sort would be the wrong way to go."[11][12]

Notes

1. Cortesi, Aldo (27 April 2007). "Visualising Sorting Algorithms" (https://corte.si/posts/code/visualisingsorting/index.html). Retrieved 16 March 2017.
2. "[JDK-6804124] (coll) Replace "modified mergesort" in java.util.Arrays.sort with timsort - Java Bug System" (https://bugs.openjdk.java.net/browse/JDK-6804124). *bugs.openjdk.java.net*. Retrieved 2020-01-11.
3. Peters, Tim (2002-07-20). "[Python-Dev] Sorting" (https://mail.python.org/pipermail/python-dev/2002-July/026837.html). Retrieved 2020-01-11.
4. "EDWARD FRIEND Obituary (2019) - Washington, DC - The Washington Post" (https://www.legacy.com/us/obituaries/washingtonpost/name/edward-friend-obituary?id=1791684). *Legacy.com*.
5. Friend, Edward H. (1956). "Sorting on Electronic Computer Systems" (https://doi.org/10.1145%2F320831.320833). *Journal of the ACM*. **3** (3): 134–168. doi:10.1145/320831.320833 (https://doi.org/10.1145%2F320831.320833). S2CID 16071355 (https://api.semanticscholar.org/CorpusID:16071355).
6. Astrachan, Owen (2003). "Bubble sort: an archaeological algorithmic analysis" (http://www.cs.duke.edu/~ola/papers/bubble.pdf) (PDF). *ACM SIGCSE Bulletin*. **35** (1): 1–5. doi:10.1145/792548.611918 (https://doi.org/10.1145%2F792548.611918). ISSN 0097-8418 (https://search.worldcat.org/issn/0097-8418).
7. "jargon, node: bogo-sort" (http://www.jargon.net/jargonfile/b/bogo-sort.html). *www.jargon.net*.
8. Donald Knuth. *The Art of Computer Programming*, Volume 3: *Sorting and Searching*, Second Edition. Addison-Wesley, 1998. ISBN 0-201-89685-0. Pages 106–110 of section 5.2.2: Sorting by Exchanging. "[A]lthough the techniques used in the calculations [to analyze the bubble sort] are instructive, the results are disappointing since they tell us that the bubble sort isn't really very good at all. Compared to straight insertion […], bubble sorting requires a more complicated program and takes about twice as long!" (Quote from the first edition, 1973.)
9. Black, Paul E. (24 August 2009). "bubble sort" (https://xlinux.nist.gov/dads/HTML/bubblesort.html). *Dictionary of Algorithms and Data Structures*. National Institute of Standards and Technology. Retrieved 1 October 2014.
10. Knuth, Donald (1997). *The Art of Computer Programming: Volume 3: Searching and Sorting*. Addison-Wesley. p. 80. ISBN 0201896850.
11. Lai Stirland, Sarah (2007-11-14). "Obama Passes His Google Interview" (https://www.wired.com/2007/11/obama-elect-me/). *Wired*. Retrieved 2020-10-27.
12. Barack Obama, Eric Schmidt (Nov 14, 2007). *Barack Obama | Candidates at Google* (https://www.youtube.com/watch?v=m4yVlPqeZwo) (Video) (YouTube). Mountain View, CA 94043 The Googleplex: Talks at Google. Event occurs at 23:20. Archived (https://web.archive.org/web/20190907131624/https://www.youtube.com/watch?v=m4yVlPqeZwo) from the original on September 7, 2019. Retrieved Sep 18, 2019.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*, Second Edition. MIT Press and McGraw-Hill, 2001. ISBN 0-262-03293-7. Problem 2-2, pg.40.
- Sorting in the Presence of Branch Prediction and Caches (https://www.cs.tcd.ie/publications/tech-reports/reports.05/TCD-CS-2005-57.pdf)

- Fundamentals of Data Structures by Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed ISBN 81-7371-605-6
- Owen Astrachan. Bubble Sort: An Archaeological Algorithmic Analysis (https://users.cs.duke.edu/~ola/bubble/bubble.html)

External links

- Martin, David R. (2007). "Animated Sorting Algorithms: Bubble Sort" (https://web.archive.org/web/20150303084352/http://www.sorting-algorithms.com/bubble-sort). Archived from the original (http://www.sorting-algorithms.com/bubble-sort) on 2015-03-03. – graphical demonstration
- "Lafore's Bubble Sort" (https://web.archive.org/web/20080119212201/http://lecture.ecc.u-tokyo.ac.jp/~ueda/JavaApplet/BubbleSort.html). Archived from the original (http://lecture.ecc.u-tokyo.ac.jp/~ueda/JavaApplet/BubbleSort.html) on 2008-01-19. Retrieved 2006-02-25. (Java applet animation)
- OEIS sequence A008302 (Table (statistics) of the number of permutations of [n] that need k pair-swaps during the sorting) (https://oeis.org/A008302)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Bubble_sort&oldid=1335367581"